

Audio Fingerprinting for Indian Classical Music

A high-level overview — concepts, pipelines, and the questions they'll ask you

Author: Aryan Banwala · DTU CSE · B.Tech-II

Generated: 2026-05-18

This doc is the friendly overview. Pipeline-focused. Concepts, not formulas. After every Part there's a "Questions they'll ask" box with the most likely viva / interview / advisor questions and ready answers.

If you want the math + derivations (NT-Xent loss, McNemar test, Wilson CIs, etc.), see the companion doc `AFP_Teaching_Guide.pdf`.

The headline. 5-system benchmark on Saraga 1.5. NAFP baseline at 98.3 % HR@1 (1 sec), recipe v3 at 99.5 % (~68 % fewer mistakes), statistically validated, published with a live calibrated demo.

Reproducibility. Every claim is grounded in a file in the repo (`data/results/...`) or a peer-reviewed paper. Pre-registration protocols are committed in git. No hallucinations.

Artifacts.

Live demo · huggingface.co/spaces/Tachyeon/afp-indian-classical-demo

Dataset · huggingface.co/datasets/Tachyeon/audio-fingerprint-indian-bench

Code · github.com/ipritamdash/afp-indian-classical

Audio Fingerprinting for Indian Classical Music

— Overview Guide

A teaching walkthrough of your project. Friendly voice, lots of analogies, plenty of "imagine this" examples. The goal isn't to make you memorize formulas — it's to make you *understand* what's happening at each step, well enough to explain it to someone else.

How this doc is organized

- Each Part teaches **one concept**, builds it from the ground up with analogies, and walks through what it looks like in practice.
- After each Part there's a "**Questions they'll ask**" box — the kinds of questions a viva examiner, advisor, or interviewer will probably throw at you, with one-paragraph answers ready.
- The last Part is a one-page summary for last-minute review.

If you want the deep math (NT-Xent loss formulas, McNemar derivation, Wilson CI math), open the companion doc `AFP_Teaching_Guide.pdf`. This doc is for **understanding**, not deriving.

Table of contents

1. [Part 1 — The whole project, in one page](#)
2. [Part 2 — What audio fingerprinting actually is](#)
3. [Part 3 — Why short clips are hard](#)
4. [Part 4 — The five systems you compared](#)
5. [Part 5 — How NAFP works \(the pipeline\)](#)
6. [Part 6 — How NAFP learns \(training\)](#)
7. [Part 7 — Augmentation: teaching the model what to ignore](#)
8. [Part 8 — NMFP and what they fixed](#)
9. [Part 9 — Saraga: your music corpus](#)
10. [Part 10 — How you built test queries \(the multiplication explained\)](#)
11. [Part 11 — How you score the systems](#)
12. [Part 12 — Seeds and why you used three](#)
13. [Part 13 — Kaggle baseline training](#)
14. [Part 14 — Recipe v3: your improvement](#)
15. [Part 15 — The headline result](#)
16. [Part 16 — Why baseline made mistakes \(the failure-mode story\)](#)
17. [Part 17 — Publishing on Hugging Face](#)
18. [Part 18 — The live demo](#)
19. [Part 19 — Threshold calibration \(open-world\)](#)

Part 1 — The whole project, in one page

Before diving in, let me give you the entire project in one shot.

You built a **benchmark** — a careful, fair comparison — of audio fingerprinting systems on Indian classical music. Audio fingerprinting is the same thing Shazam does: take a short audio clip, identify which song it's from. The technical question: *how well do different systems do this on Indian classical music specifically?*

Why Indian classical specifically? Because most fingerprinting research has been done on Western pop. Indian classical has very different acoustic characteristics — long slow vocal sections (alaap) with few percussive beats, the continuous low drone of a tanpura, and an interesting "same-artist hub" effect where multiple recordings by the same artist start to look similar to fingerprinting systems.

You tested **five** systems on the same data:

- **Olaf, Dejavu, Panako** — the old-school approach, using hand-engineered hash codes from spectrogram peaks (descendants of the original Shazam algorithm from 2003)
- **NAFP** — a 2021 deep learning approach, a small neural network trained with "contrastive learning"
- **NMFP** — a 2025 paper that improved NAFP's training recipe

Your **finding**: NAFP works well on Indian classical (98.3% accurate at 1-second clips on the main test set), but you spotted a specific failure pattern — it confuses different recordings *by the same artist*. So you applied two of NMFP's training fixes to NAFP, retrained the model on Colab three times with different random seeds, and showed:

- **99.5% accuracy** at 1-second clips (~70% fewer mistakes)
- **~68% fewer mistakes** across all 8 evaluation slices
- The improvement is **statistically real** — chance of luck is roughly 1 in 300,000
- Everything is **published**: code on GitHub, results on a Hugging Face dataset, and a live demo where anyone can try the system

That's the whole thing. Now let's unpack each piece in the order you'd want to learn it.

Questions they'll ask

Q: Can you give me the one-sentence pitch? "I built a 5-system audio fingerprinting benchmark on Indian classical music, found that the dominant failure mode is same-artist confusion, and demonstrated a statistically-significant ~68% reduction in errors using two training-recipe fixes from a newer paper — with everything reproducible and live."

Q: Why is this useful? Most audio fingerprinting papers test on Western pop. Indian classical is acoustically different — long alaaps, tanpura drones, same-artist hubs. Showing what works (and what doesn't) on this genre extends the practical reach of these systems.

Q: What's novel? The benchmark itself (no one had run all 5 systems on Saraga before), the same-

Part 2 — What audio fingerprinting actually is

Let me build this from scratch, step by step.

The Shazam example, slowly

You're at a café. A song comes on that you've never heard but you love it. You pull out your phone, open Shazam, hold the phone up to the speaker. About 5 seconds later, Shazam shows you the song title and artist.

What just happened?

Step 1: Your phone recorded 5 seconds of audio. **Step 2:** A small algorithm on your phone converted that 5 seconds into a tiny digital signature — basically a compact summary of "what this audio sounds like." **Step 3:** Your phone sent that signature (not the audio) to Shazam's server. **Step 4:** Shazam's server compared your signature against a database of ~100 million signatures, one for each song in their library. **Step 5:** The closest match was returned to your phone.

That's audio fingerprinting. The trick is in **Step 2** — how do you convert 5 seconds of messy real-world audio into a tiny signature that *uniquely identifies the underlying song*?

Why this is hard

Imagine I gave you a 5-second clip from a song. Now I give you a noisy version of the same 5 seconds — recorded through a cheap microphone in a café with people talking in the background. The two clips sound *different* to your ears (one is clean, one is noisy), but you'd say "yeah, that's the same song."

A good fingerprinter has to agree with you. It has to produce **the same signature** for both versions, even though the actual audio waveforms are different.

That's the magic trick: **produce identical fingerprints for clean and noisy versions of the same recording, but different fingerprints for clean versions of two different recordings.**

The fingerprint analogy

This is exactly like your physical fingerprint:

- **Much smaller than the whole thing** (your fingerprint is 10 KB compared to your whole body)
- **Unique enough to identify** (no two people share fingerprints — well, almost no two)
- **Robust to imperfections** (a slightly smudged fingerprint still identifies you)
- **A one-way summary** (you can't reconstruct a person from their fingerprint)

Audio fingerprints have the same four properties. They're tiny, unique, noise-tolerant, and you can't get the original audio back from them.

The three goals that fight each other

Every fingerprinter is trying to satisfy three goals simultaneously:

Goal	What it means	What happens if it fails
Discriminative	Different recordings → different fingerprints	Everything matches everything else
Invariant	Same recording with noise → same fingerprint	Real-world queries don't work
Fast	Search millions of fingerprints in < 1 second	Users give up

Notice that these goals **fight each other**. A super-discriminative fingerprint (one that distinguishes every tiny audio difference) won't be invariant to noise. A super-invariant fingerprint (one that ignores everything except the "essence") won't discriminate similar recordings. Finding the right balance is the engineering challenge.

Two big families of fingerprinters

There are essentially two ways to build the magic Step 2 algorithm:

Family 1 — Hand-engineered hashing (the old way): Look at the spectrogram (a 2D picture of the audio — frequency vs time), find the loudest peaks, encode pairs of peaks as integer "hash codes." Store all the hashes in a database. Match by hash lookup.

This is what Shazam did in 2003. It's still good. Olaf, Dejavu, and Panako are modern open-source descendants.

Family 2 — Neural networks (the new way): Train a neural network to compress audio into a fixed-size vector of numbers. The network *learns* what features matter by being shown lots of examples.

NAFP (2021) and NMFP (2025) are in this family.

Family 1 is simpler, faster, no training needed. Family 2 is more accurate, especially on short clips, but needs training compute.

Closed-world vs open-world

One more concept before we move on. There are two ways audio fingerprinting can be tested or deployed.

Closed-world means *the truth is always in the library*. You're handed a clip from a Saraga track and asked to identify which one — you know one of the 357 Saraga tracks is the right answer; the system just has to pick the right one. This is what your **benchmark** does. It's easier.

Open-world means *the truth might not be in the library*. You're handed any random audio — could be a Saraga track, could be Bollywood, could be a voice memo, could be white noise. The system has to find the match if there is one, OR say "I don't know" if there isn't. This is what your **live demo** does. It's harder because the system needs to know when to give up.

The benchmark and the demo solve different problems with the same model. The demo adds a confidence threshold (Part 19) to handle the "I don't know" case.

Questions they'll ask

Q: Why convert audio to a signature instead of just storing the audio? Three reasons. (1) Signatures are tiny — kilobytes vs megabytes. (2) Searching signatures is fast. (3) The transformation is one-way, so storing fingerprints doesn't redistribute copyrighted audio.

Q: Why is this called "fingerprinting" and not "matching"? The fingerprint metaphor captures the four properties: small, unique, robust, one-way. "Matching" is just one half of what's happening; the other half is *creating* a compact identifier.

Q: Could you do this just by storing song titles? Only if you already know the title. The whole point of fingerprinting is identifying audio *when you don't have the metadata*. Like recognizing a face when you don't know the name.

Part 3 — Why short clips are hard

This part justifies a lot of what comes later. Worth understanding why short audio is the hard case.

Information content scales with length

Imagine I show you a one-pixel image and ask "what does this depict?" You can't tell — one pixel is just a color. Show me a 100-pixel image and you might guess it's a face. Show me a 1000-pixel image and you can probably tell who the face belongs to.

Audio works the same way. A 1-second clip might contain only 2-3 musical notes. A 10-second clip contains rhythm, melody, an arc of harmonic progression, maybe a phrase boundary. There's just more **information** in longer audio.

How this affects classical fingerprinters

Hash-based systems work by finding peaks in the spectrogram (loud points) and encoding pairs of peaks as hash codes. Then they look up the hashes in a database and vote on the best match.

Imagine you're playing a guessing game. Someone gives you 5 random words from a book and you have to figure out which book. With 5 words, you've got *some* chance. Now they give you 500 random words — easy, you can identify the book with confidence.

That's what's happening with hash-based fingerprinters:

- 1-second clip: ~10 peaks → ~50 hashes → "5 words" worth of evidence
- 10-second clip: ~100 peaks → ~500 hashes → "500 words" worth of evidence

So Olaf gets 49% at 1 second but 99.8% at 10 seconds. The algorithm is the same; only the amount of evidence changed.

How this affects neural fingerprinters

Neural networks are more robust to short clips because they're trained to compress *whatever* audio they see into a fingerprint that captures the audio's character — timbre, harmonic structure, micro-rhythm. Even from 1 second, the model can extract more than just peak positions.

But even neural networks suffer somewhat. NAFP baseline gets 98.3% at 1 second vs 100% at 10 seconds. Most of that 1.7% miss rate is the same-artist confusion problem (Part 16) — at 1 second, there's not enough information to tell two recordings by the same artist apart.

Why we test at 4 lengths

Your benchmark tests at 1, 3, 5, and 10 seconds. Not because each length is equally interesting, but because **the differences between systems show up most at short lengths**. By 10 seconds, every neural system scores 100% — there's no signal. By 1 second, the spread is huge:

System	1 second	10 seconds
Panako	0.0%	99.7%
Olaf	49.2%	99.8%
Dejavu	74.5%	100%
NAFP baseline	98.3%	100%
Recipe v3	99.5%	100%

At 10 seconds everything looks the same. At 1 second the systems are clearly ranked. That's why the 1-second column is the **headline cell** for comparison.

Questions they'll ask

Q: Why care about 1-second clips? Real users record longer. Phone-mic Shazam queries are typically 5-15 seconds, you're right. But 1-second is the *adversarial* test — the limit case. If a system passes 1-second testing, you know it has headroom for noisier or shorter real-world conditions.

Q: Why not test at 0.5 seconds? NAFP is trained on 1-second windows. Going shorter would test something the model wasn't designed for. 1 second is the smallest *natural* unit.

Q: At 10 seconds everything is at 100%. Why even include 10 seconds? Sanity check. If a system underperforms at 10 seconds, something is broken. Plus it lets you say "this system saturates at 5 seconds and above" which is a meaningful capability statement.

Part 4 — The five systems you compared

Let me walk through each one with a little story for each.

Olaf — the modern Shazam in C

Joren Six (a researcher at Ghent University) reimplemented the original 2003 Shazam algorithm in modern C around 2023. Small footprint, blazing fast, uses LMDB (a tiny embedded database) for storage.

What it does, step by step:

1. Convert the audio's waveform into a spectrogram (the frequency-vs-time picture)
2. Find peaks — the loudest pixels in small local neighborhoods of the spectrogram
3. For each peak, look at the next few peaks in time. Encode each (peak1, peak2, time-difference) as a 32-bit hash code
4. Store all the hashes in LMDB along with which track they came from

5. To match a query: extract its hashes, look them up, vote on the most likely answer

The intuition for *why this works*: Two peaks that are loud enough to be selected are real musical events (a note attack, a drum hit), not noise. Two adjacent loud peaks tend to stay together even if you add background noise or change the EQ. So matching *pairs* is much more robust than matching individual peaks.

Performance on your benchmark: Half the 1-second clips fail to match. At 10 seconds it's basically perfect.

Dejavu — Olaf's Python cousin

Will Drevo wrote Dejavu in 2015 in Python, using PostgreSQL as the database. **Same algorithm as Olaf** in spirit — peak-pair hashing. Different implementation choices: PostgreSQL instead of LMDB, more aggressive peak finding (more peaks per second), wider "fan-out" per anchor (more pair partners considered).

These small parameter differences make a real difference at 1 second. Dejavu finds more peaks → produces more hashes → gets more votes → wins more often. **75% at 1 second vs Olaf's 49%.**

It's the same family of approach, just better tuned. Don't make too much of the difference between Olaf and Dejavu — they're cousins, not rivals.

Panako — the CQT triplet system

Joren Six again (same guy who made Olaf), but in 2014 with a different transform.

Instead of a regular spectrogram, Panako uses the **Constant-Q Transform (CQT)**. Without diving into math: a normal spectrogram has equal-frequency bins (like 0-22 Hz, 22-44 Hz, ...). A CQT has equal-musical-pitch bins (one bin per semitone — C, C#, D, D#, ...). The CQT is "musical" in a way regular spectrograms aren't.

Then Panako picks **triplets** of peaks (three peaks at a time) instead of pairs. Each triplet gets a hash code. The clever part: the hash encodes ratios that are invariant to **time-stretching** (playing the audio faster or slower keeps the same triplet hash).

This makes Panako designed for forensic use cases like "did this song appear in this movie, possibly time-stretched?"

The problem on short clips: Triplets need three peaks close enough in time. A 1-second clip rarely has three loud peaks close enough together. So Panako produces *zero* triplets on 1-second clips, has nothing to match, and returns nothing. **0.0% at 1 second.**

Panako only starts working at 5 seconds (92.2%) and is excellent at 10 seconds (99.7%). It's the right tool for forensic-length queries, the wrong tool for Shazam-style queries.

NAFP — the neural baseline you started with

The first non-classical system, and the one you're improving.

NAFP (Chang et al., ICASSP 2021) is a small convolutional neural network — 19 million parameters, fits on a phone. It takes 1 second of audio as input and outputs a 128-number vector (the "fingerprint").

The network was trained using **contrastive learning** (Part 6 explains this in depth): show it pairs of audio

that should produce similar fingerprints, and pairs that should produce different fingerprints. Let it learn what features matter.

Performance on your benchmark: 98.3% at 1 second. That's a massive jump from Dejavu's 75%. Neural fingerprinting clearly wins at short clips.

But there's a 1.7% miss rate at 1 second — 17 mistakes per 1000 queries. Your project investigates what those mistakes are and how to fix them.

NMFP — the ceiling reference

NMFP (Araz et al., ISMIR 2025) is the most recent paper in this line. They took NAFP's exact architecture and applied 5 training recipe fixes. Same model size, same network shape — just trained better.

Results: NMFP scores **100% on every single one of your 8 evaluation cells**. Zero mistakes anywhere.

That's the upper bound — the best anyone has shown can be done on this benchmark.

A subtle point: Your work doesn't try to *beat* NMFP. They used ~10× your training compute. Your work shows that applying *just 2 of their 5 fixes* (the cheap ones) gets you ~95% of the way to their ceiling at ~10% of their compute.

The big table, all together

System	1 sec	3 sec	5 sec	10 sec
Olaf	49.2%	95.5%	99.3%	99.8%
Dejavu	74.5%	96.9%	99.4%	100%
Panako	0%	0%	92.2%	99.7%
NAFP (baseline)	98.3%	99.8%	99.9%	100%
Recipe v3 (your work)	99.5%	100%	100%	100%
NMFP (ceiling)	100%	100%	100%	100%

The story this table tells in plain English:

- Hash-based systems are fine at long clips, weak at short clips.
- Panako has a hard floor — doesn't work below 5 seconds.
- Neural systems win at every length.
- Recipe v3 closes most of the gap from baseline (98.3%) to the NMFP ceiling (100%) at the hardest length.

Questions they'll ask

Q: Why is Panako so bad at 1 second? Because Panako matches triplets (three peaks). A 1-second clip rarely contains three peaks close enough together to form a triplet. Without triplets, Panako has nothing to match. This is a fundamental limit of the triplet approach on short audio — not a bug.

Q: Why include Panako if it fails so badly? Because Panako is a real, used system in forensic AFP communities. Showing empirically that it doesn't work on short clips is itself a useful finding — it helps

people choose the right tool for their use case.

Q: Did you tune the hyperparameters of each system or use defaults? Defaults. The point of a fair benchmark is "how do these systems perform as you'd actually use them?" — not "how do they perform after 100 hours of hyperparameter optimization on each."

Part 5 — How NAFP works (the pipeline)

Now we get to the model you actually improved. Let me walk you through what happens inside it, step by step, with no math.

The whole pipeline in a picture



Five stages. Let me explain what each one does and *why*.

Stage 1 — kapre log-mel front-end (audio → spectrogram)

The model can't work directly on a sound wave. A sound wave is a 1D wiggle: 8000 numbers per second representing the air pressure over time. You can't see the song in that wiggle.

So the first stage is **converting the wiggle into a picture** — a spectrogram. This is the same picture you've seen on YouTube videos or in audio editors: time goes left to right, frequency goes bottom to top, brightness shows how loud each frequency is at each moment.

The "log-mel" part is a refinement: humans don't hear frequency linearly. We hear pitch as logarithmic — the difference between 100 Hz and 200 Hz "sounds like" the same step as 1000 Hz to 2000 Hz. Both are an octave. So instead of a linear frequency axis, we use the "mel scale," which is logarithmic-ish and matches human hearing. The "log" part squashes the brightness range so quiet and loud parts are both visible.

Output of Stage 1: a 256-row by 32-column image. Each row is a frequency band, each column is a time slice. Brighter pixels = louder frequency at that time.

This image is what the rest of the model "sees." Think of it as the audio's photograph.

Stage 2 — CNN backbone (looking at the picture)

A **Convolutional Neural Network (CNN)** is the standard tool for image recognition. ImageNet, face recognition, medical imaging — all CNNs. NAEP applies the same tool to audio spectrograms.

How a CNN works in plain English: imagine you're trying to recognize what's in a photo. You don't look at every pixel individually — you look at small patches and ask "does this patch look like an edge, a curve, a fur pattern?" Then you combine those patch-features into larger features, then larger again, until eventually you have a representation of the whole image.

That's what a CNN does mechanically. Slide small filters (like 3×3 patches) over the image, then bigger patches over the result of that, and so on. After 8 such "convolution blocks," NAEP has compressed the 256×32 spectrogram into a single 1024-number vector representing the "essence" of that audio clip.

The 19 million parameters of the model are the *learned* values of the filters. Each filter is essentially a learned pattern detector — one might detect "sustained tone in mid frequencies," another might detect "percussion attack." The model figures out which patterns are useful during training.

Stage 3 — DivEnc projection head (1024 → 128)

You've got a 1024-number summary. That's big — too big for fast search across 690,414 reference fingerprints. You want it compressed to 128 numbers.

You could just multiply by a 1024×128 matrix and call it done. But that lets the model dump all the useful information into a few dimensions and leave the rest as noise.

DivEnc ("Divide and Encode") is a smarter compression. Take the 1024-number vector, split it into 128 groups of 8 each, run a tiny neural network on each group, concatenate the 128 outputs into a single 128-number vector.

The clever part: each output number only "sees" 1/128 of the input. So the model can't dump everything into one output dimension — it has to spread information evenly across all 128 dimensions. Every dimension carries real information, not noise.

Output of Stage 3: a 128-number vector — your candidate fingerprint.

Stage 4 — L2 normalize (the final polish)

The last step: divide every fingerprint by its own length so the length becomes exactly 1.

What does that mean visually? Imagine the 128-number vector as an arrow in 128-dimensional space. Without normalization, arrows have all sorts of lengths. With L2 normalization, every arrow has length exactly 1 — they all sit on the surface of an imaginary "unit sphere" in 128D.

Why is this useful?

Two reasons:

1. **Comparing fingerprints becomes super fast.** When two vectors both have length 1, similarity = dot product, which is a single matrix multiply that FAISS (your search library) can do in microseconds.
2. **The training loss function works best on unit-length vectors.** Without normalization, the model could just cheat by inflating vector magnitudes instead of learning useful features.

Output of Stage 4 (the final fingerprint): a 128-number unit-length vector that represents the audio.

Putting it all together — the inference flow

Here's what happens when you use the trained model:

Pre-computing the library (done once, ahead of time):

```
For each Saraga track:
    Slice the audio into 1-second windows (sliding 0.5 sec at a time)
    For each window, run it through NAFP → get a 128-number fingerprint
    Store the fingerprint in a NumPy memmap file

You end up with 690,414 fingerprints total (across all 357 tracks)
Total disk: ~354 MB
```

Searching for a match (every time a user queries):

```
Take the user's audio query
Slice into 1-second windows (sliding 0.5 sec)
Run each through NAFP → query fingerprints
For each query fingerprint, ask FAISS: "which library fingerprint is closest?"
Group results by track, average scores, return the winner
```

The whole search takes ~50 milliseconds total on your Mac. Real-time.

Questions they'll ask

Q: Why 128 numbers? Why not 64 or 256? The NAFP paper chose 128 by ablation — it's a good balance between size (smaller is faster) and discrimination (bigger captures more nuance). At 64, accuracy drops noticeably. At 256, accuracy is similar but storage and search both double. 128 is the sweet spot.

Q: Why a CNN and not a transformer? Three reasons: (1) CNNs are smaller — 19M params vs 100M+ for typical transformers. (2) The NAFP paper showed CNNs work well for this task. (3) Smaller models train faster on free GPUs (Kaggle, Colab) and are easier to deploy on phones.

Q: What's the difference between fingerprint and embedding? Same thing, different name.

"Embedding" is the ML term for any learned vector representation. "Fingerprint" is the application-specific term for an embedding used to identify content. Two names, one concept.

Q: Does the spectrogram capture the full audio? No — phase information is discarded (only magnitude is kept). But for identification, magnitude is enough. You couldn't reconstruct the original audio from the spectrogram, which is actually a *good* thing for copyright reasons.

Part 6 — How NAFP learns (training)

The architecture is half the story. The other half is *how the network learns to produce good fingerprints*. This is where contrastive learning comes in.

The teaching analogy

Imagine I'm teaching a kid to recognize their cousin Alex. I show them 100 photos:

- Alex in sunny light
- Alex in a baseball cap
- Alex with a different haircut
- Alex from the side
- Alex looking serious
- ...

I also show them 100 photos of *other people* — strangers, friends, neighbors.

For each photo I say: "This is Alex" or "This is not Alex."

After enough photos, the kid figures out which features make Alex *Alex* — facial structure, eye color, the curve of the nose — things that are stable across haircuts and lighting. And they learn what *isn't* Alex — anyone else.

That's contrastive learning. Show many "same" pairs and many "different" pairs; let the model figure out which features distinguish them.

How NAFP does this for audio

Same idea, but for audio clips instead of faces.

Step 1: Take 1 second of audio from a track. Call it the **anchor**.

Step 2: Make a "noisy version" of the anchor — call it the **positive**. Same audio, but distorted in realistic ways:

- Slight time shift (start 0.3 sec earlier)
- Mixed with background noise (subway, café)
- Convolved with a room's reverb signature
- A rectangular patch of the spectrogram zeroed out (SpecAugment)

The positive *sounds different* from the anchor but is from the same recording.

Step 3: Grab 1 second of audio from a *different* track. Call it the **negative**.

Step 4: Feed all three through the model. The model produces three 128-number fingerprints.

Step 5: Apply the loss function. The loss says:

- Anchor and positive should be *similar* (their fingerprints should point in roughly the same direction)
- Anchor and negative should be *different* (their fingerprints should point in different directions)

Step 6: Compute gradients, update the model's weights slightly so it does better next time.

Repeat this for millions of (anchor, positive, negative) triples.

What a training batch looks like in practice

In practice you don't just use one positive and one negative per anchor. You use a whole batch:

- Pick 160 anchor segments (each from a different track)
- For each anchor, make a positive (same track, augmented)
- You now have 320 segments total in the batch (160 anchors + 160 positives)
- For each anchor, the positive is the "right answer" and the OTHER 318 segments are negatives

The loss pushes each anchor close to its own positive and away from the other 318 segments. With 318 negatives per training step, the model gets a lot of "what's NOT this anchor" information per step.

This is why bigger batches help. NMFP used batches of 1536 (more negatives). Your recipe v3 uses 320 (max your Colab GPU can hold).

The NT-Xent loss function

The actual loss function NAFP uses is called **NT-Xent** (Normalized Temperature-scaled Cross-Entropy).

Without diving into the formula:

- It's a "soft" version of "pull the positive close, push everything else away"
- A **temperature parameter** ($\tau = 0.05$) controls how strict the discrimination is
- Low temperature = strict discrimination (only very-close pairs count as "similar")

For now just remember: **NT-Xent is the math that turns "pull positives, push negatives" into a number the model can optimize.**

One sneaky problem: same-track false negatives

Here's a subtle bug in the default NAFP training: the random sampler can pick two segments from the **same track** for the same batch (say, second 10 and second 80 of the same recording).

NT-Xent treats them as *negatives* of each other — pushes them apart. But they're from the **same recording**! They should be *similar*! The loss is sending the wrong signal.

This is a false negative in the training data. It hurts the model's ability to learn proper invariance.

The fix (NMFP fix #2, used in recipe v3): the `random_oneshot` sampler. Pick at most ONE segment per track per batch. Reshuffle which segment each epoch. No false negatives. Cleaner training signal.

This is one of the two changes you applied for recipe v3. Even though it's just a sampler change, it has a measurable effect on the model — especially on Indian classical music, where same-track segments are particularly similar (same voice, same room, similar musical phrasing).

How long is training?

For your baseline NAFFP: 10 epochs (10 passes through 10,000 training tracks) at batch size 120. Total wall time on a free Kaggle T4 GPU: about 45 minutes.

For recipe v3: 30 epochs at batch size 320. Total wall time on a free Colab L4 GPU: about 27 minutes per seed, 80 minutes for three seeds.

Questions they'll ask

Q: What is the "loss function" trying to minimize? The loss is a single number that summarizes "how wrong is the model right now?" Lower is better. Training is gradient descent — taking tiny steps in the direction that lowers the loss. After enough steps, the loss is small and the model is good.

Q: Why does contrastive learning work without explicit labels? Because it implicitly creates labels through pairing. The anchor-positive pair is labeled "same"; the anchor-negative pair is labeled "different." The model learns "what makes audio identifiable" — which transfers to any kind of audio it might see later.

Q: Why train on Western music if you want to identify Indian classical music? Because the *task* — "produce a representation invariant to noise but discriminative across recordings" — is genre-agnostic. The features the model learns (timbre, harmonic structure) are universal. It transfers across genres without retraining.

Q: Is FMA-medium really enough training data? Why not more? 10,000 tracks is enough because of contrastive learning. Each pair (anchor, positive) is a training example. With augmentation, each track generates effectively thousands of unique anchor-positive pairs. Total effective training examples are in the tens of millions, not the thousands.

Part 7 — Augmentation: teaching the model what to ignore

The augmentation pipeline is *critical* to making the model work in the real world. Without it, the model just memorizes the training audio. With it, the model learns what's noise vs what's signal.

The role of augmentation

When you make a positive for an anchor, you don't just hand the model a slightly-time-shifted version. You aggressively distort it — add noise, simulate reverb, zero out spectrogram patches. The harder the distortion, the more the model has to learn to *ignore that distortion* in order to still match the anchor.

Think of it like training an athlete on harder conditions than they'll face in competition. Sprinters train with parachutes attached so they're faster when the parachute comes off.

The four augmentations NAFP uses

1. Time offset shift The positive is sampled at a slightly different starting offset than the anchor (e.g., the anchor is at 12.0 sec, the positive is at 12.4 sec). The model learns that small time shifts shouldn't change the fingerprint much.

2. Background noise mixing Real-world recordings are usually clean. But a query through a phone mic captured in a café will have background noise (people talking, espresso machine, traffic).

NAFP uses the **TUT 2016** dataset — real recordings of urban environments. For each positive, the trainer:

- Picks a random noise clip
- Computes a target signal-to-noise ratio (SNR) between 0 and 10 dB
- Mixes the noise into the audio at that ratio

SNR 0 dB = noise as loud as music. SNR 10 dB = noise 1/10 as loud as music. The model sees both extremes during training.

3. Impulse response convolution (room reverb) A real-world query isn't just noisy — it has reverb. If you play music in a room, the room's walls bounce the sound back; your microphone captures the original + the bounces.

To simulate this, NAFP uses **impulse responses (IRs)** — short recordings of a single clap played in a real room. When you "convolve" audio with an IR, you simulate playing that audio in that room.

NAFP uses one IR dataset. NMFP uses four. More variety = more invariance learned.

4. SpecAugment (spectrogram cutout) A weirder augmentation, borrowed from speech recognition. After computing the log-mel spectrogram, zero out a random rectangular patch — both a band of frequencies and a span of time.

Why? It forces the model to make predictions from *partial* spectrograms. So if part of a query is corrupted (clipped, drowned out), the model still works.

What NAFP deliberately does NOT augment

Two distortions the paper authors decided to *not* apply:

- **Pitch shifting** — would teach the model "the same song shifted up by a semitone is the same song." But we *want* to distinguish songs from each other. If two different songs happened to be related by pitch shift, the model would conflate them. Bad augmentation.
- **Time stretching** — same problem. Two different tempo versions of the same song should match; two different songs shouldn't be conflated just because of tempo. Bad augmentation.

These are *forensic* properties (Panako has them because forensic search wants them). NAFP/NMFP are tuned for Shazam-style identification, where pitch and tempo invariance are not wanted.

Why this matters for Indian classical

Indian classical recordings have specific real-world characteristics:

- A continuous tanpura drone (low-frequency hum)
- Audience noise during concerts
- Variable room reverb (different concert halls)

- Different microphone placements

Every one of these is simulated by NAFP's training augmentations:

- Tanpura drone → handled because of low F_{MIN} + augmentation
- Audience noise → handled by background noise augmentation
- Room reverb → handled by IR augmentation
- Mic differences → handled by SpecAugment + all of the above

So even though NAFP was trained on Western music, its augmentations are general enough that the model handles Indian classical real-world conditions. This is the *cross-domain transfer* mentioned earlier.

Questions they'll ask

Q: How aggressive is "aggressive augmentation"? SNR 0 dB means noise is as loud as the music — to a human, the music is almost lost in the noise. Yet the model is supposed to still recognize which recording it's from. That's aggressive.

Q: What if a real-world query has a distortion you didn't train on? Probably some accuracy loss. The strength of contrastive learning is generalization — features learned to be invariant to one kind of noise tend to be invariant to similar kinds of noise. But there are limits.

Q: Could you also augment in spectrogram domain instead of audio domain? SpecAugment is exactly that. The others (noise, IR) need to be in audio domain because they involve physical mixing/convolution.

Part 8 — NMFP and what they fixed

NMFP is the 2025 paper that took NAFP's architecture and made it work much better through training fixes alone. Let me walk through what they changed and why.

The big finding of NMFP

The architecture is fine. The original NAFP paper's bottleneck is **the training recipe**, not the model. By fixing 5 things in *how* the model is trained — same architecture, same network shape — NMFP gets substantially better results.

This is a powerful finding in ML. Often "better results" don't require new models. They require careful attention to the training process.

The 5 fixes (in plain English)

Fix #1 — More diverse noise and reverb datasets NAFP uses one noise dataset (TUT 2016) and one IR dataset. NMFP uses **four**: TUT 2016 + OpenAIR + MIT + AIR. More variety in the augmentation → the model has to learn invariances over a wider range → better real-world robustness.

(You did NOT apply this in recipe v3 because it requires modifying the dataset loader. Future work.)

Fix #2 — One-anchor-per-track sampler The same-track false negative fix I explained in Part 6. The default NAFP sampler can put two segments from the same track into one batch, which the loss function

then incorrectly pushes apart. The fix: pick at most one segment per track per batch.

You did apply this in recipe v3.

Fix #3 — Full impulse response duration NAFP truncates IRs to a fixed length (say, 0.5 sec) to save compute. But real rooms have longer reverb tails — sometimes seconds long. By using the full IR, the model learns to handle more reverberant environments.

(You did NOT apply this — requires modifying the augmentation pipeline. Future work.)

Fix #4 — 1-second acoustic history For each 1-second anchor, NMFP additionally provides 1 second of audio that *preceded* the anchor as context. The model can use this past context to disambiguate the anchor.

(You did NOT apply this — requires modifying the model's input layer. Future work.)

Fix #5 — Lower frequency cutoff (F_MIN: 300 → 160 Hz) NAFP's default mel filterbank starts at 300 Hz — anything lower is discarded. But there's important content below 300 Hz: bass instruments, low vocal fundamentals, drones. For Indian classical, this is especially relevant — tanpura drones (100-200 Hz), male vocal fundamentals (80-200 Hz), tabla bass.

NMFP drops F_MIN to 160 Hz, capturing this low-frequency information.

You did apply this in recipe v3.

Plus a loss change

NMFP also switched the loss function from NT-Xent to **Triplet loss with semi-hard mining**. Triplet loss is similar in spirit (pull positives, push negatives) but uses an explicit margin and picks the hardest negative per anchor rather than averaging.

This gives an additional ~2-4% gain in NMFP. You **kept** NT-Xent in recipe v3 — partly to isolate the effect of the other changes, partly because Triplet loss requires more complex sampling code.

Summary table

#	Fix	Recipe v3?
1	Better noise + IR datasets	✗ Future work
2	One-anchor-per-track sampler	✓ Applied
3	Full IR duration	✗ Future work
4	Acoustic history	✗ Future work
5	F_MIN = 160 Hz	✓ Applied
+	Triplet loss with mining	✗ Future work

You applied 2 of 5 (the cheap ones, configuration-only). The other 3 + loss change are all "future work" for a follow-up project.

How NMFP performs on your benchmark

NMFP's published checkpoint-100 (their fully-trained model) scores **100% on every cell of your**

benchmark. Zero misses. That's the ceiling.

Your recipe v3 reaches 99.5% on the hardest cell — within 0.5% of NMFP's ceiling at ~10% of their training compute (80 min vs ~13 hours).

License caveat (important practical point)

NMFP's published weights are licensed **GPLv3 / AGPLv3** — viral copyleft. You can run their model for benchmark comparison, but you **cannot distribute it as part of an MIT-licensed product**. So your deployed demo (which is MIT licensed) ships YOUR model (recipe v3, trained from scratch on FMA-medium with the Recipe v3 changes), not NMFP's. NMFP is the ceiling reference only.

Questions they'll ask

Q: Why didn't you apply the other 3 fixes? Time and code complexity. F_MIN and the sampler are single-line changes. The other 3 require modifying the data loader, augmentation pipeline, and model input — significantly more code to write, debug, and validate. For B.Tech-II scope, the cheap fixes that already gave Bonferroni-significant improvement were the right choice.

Q: Why keep NT-Xent and not switch to Triplet? To isolate the effect of the recipe changes from the loss change. If you'd changed F_MIN + sampler + batch size + epochs + loss all at once, you couldn't tell which contributed. By keeping the loss constant, the gain you see comes specifically from the recipe.

Q: Is your work just "lite NMFP"? Partly yes — you applied 2 of their ideas. But your novel contributions are: (a) the Saraga benchmark setup (no one had done it before), (b) the same-artist failure-mode analysis (genre-specific finding), and (c) the calibrated open-world demo. Those parts are yours.

Part 9 — Saraga: your music corpus

Time to talk about the audio you're working with. The choice of corpus matters a lot — bad corpus = meaningless benchmark.

What Saraga is

Saraga 1.5 is a publicly-released corpus of Indian classical concert recordings. Published by the **CompMusic** research group at MTG, Universitat Pompeu Fabra (Barcelona, Spain).

Key facts:

- **357 tracks** total
- **108 Hindustani** (North Indian classical) — ~52 hours of audio
- **249 Carnatic** (South Indian classical) — ~96 hours of audio
- All recordings are **professional concert recordings** (multi-mic, high quality)
- Available on **Zenodo** (DOI: 10.5281/zenodo.4301737)
- License: **CC-BY-NC-SA 4.0** (free for non-commercial use)

Why Saraga specifically

Why this corpus and not something else? Four reasons:

1. **It's *the* standard public corpus for Indian classical music research.** When you cite Saraga, every Indian classical music researcher knows exactly what data you're talking about.
2. **It has rich metadata.** Each track is annotated with the artist, the raaga (melodic mode), the taala (rhythm cycle), MBIDs (canonical music identifiers), and **section boundaries** (when the alaap ends, when the composed part starts, etc.).
3. **It's diverse.** 357 tracks across two traditions, dozens of artists, many raagas.
4. **It's bounded.** 357 tracks is small enough to evaluate exhaustively but large enough to have statistical meaning.

Quick primer on Indian classical music

You don't need to be a music theorist to do this project, but a few concepts will keep coming up:

Raaga is a melodic framework — a specific scale plus rules about which notes are emphasized, which transitions are allowed, what mood the raaga evokes. There are hundreds. Each raaga has a "feel" — Mohanam is bright, Bhairavi is contemplative, Yaman is romantic. Different raagas should produce different audio signatures.

Taala is the rhythmic cycle. Common ones include teentaal (16 beats) and adi taala (8 beats). The percussion accompaniment lays out the taala.

Tonic (Sa) is the reference pitch of a performance. Crucially, it varies between recordings — one singer might use Sa at 220 Hz, another at 165 Hz. So "the same raaga" in two recordings can be at different absolute frequencies.

Alaap is the slow, free-rhythm opening of a Hindustani performance. The singer explores the raaga gradually, with no tabla. Very few percussive transients. This is hard for hash-based fingerprinters because there aren't many peaks.

Composed section is the main piece — a "bandish" (Hindustani) or "kriti" (Carnatic). Has full rhythmic accompaniment, strong attacks. Easier for AFP.

Tani avartanam is a percussion solo, Carnatic-specific. Lots of strong transients.

These section types matter because your ablation set is built to test performance per-section.

What you DON'T get with Saraga

A few things to keep honest:

- **Audio that's been re-encoded through phones.** Saraga is studio/concert audio — clean. To test phone-mic robustness, you'd need to record Saraga playback through phones, which is future work.
- **Bollywood / film music.** Different genre, different copyright situation, not in Saraga.
- **Millions of tracks.** Saraga has 357. Production AFP (Shazam) has 100M+. Your benchmark's small library size is a documented limitation.

File layout in your repo

```

data/saraga/
├─ hindustani/
│   └─ <track_id_1>/
│       └─ <track_id_1>.mp3          ← the mixed audio
│       └─ <track_id_1>.meta.json ← artist, raaga, etc.
│       └─ ...
│   └─ ... (108 tracks total)
└─ carnatic/
    └─ ... (249 tracks total)

data/refs.parquet          ← 357-row consolidated index

```

The `refs.parquet` file is what every system reads to know "these are the 357 reference tracks." It has columns: `ref_id`, `subcorpus` (hi/ca), `audio_path`, `artist`, `raaga`, `taala`, `work_mbid`, `recording_mbid`, `duration_sec`.

Questions they'll ask

Q: Only 357 tracks? Shazam has millions. Yes — Saraga is the entirety of public, properly-annotated Indian classical music. Building a million-track corpus would take years. For demonstrating a benchmark + improvement, 357 is enough. Library size limitation is honestly documented.

Q: Why aren't there more Indian classical music corpora? Several reasons: copyright complications (much Indian classical is recorded by commercial labels with strict rights), the niche size of the research community, and the difficulty of getting rich annotations (raaga/taala/section boundaries require expert annotators).

Q: Is it OK to compare across Hindustani + Carnatic in one benchmark? Yes, with care. They're distinct musical traditions but share the same underlying concepts (raaga, taala, etc.). You sample 500-500 from each so neither dominates the test set. Your headline numbers are reported per-corpus too.

Part 10 — How you built test queries (the multiplication explained)

This is the part that confused you in the earlier draft. Let me walk through it slowly with concrete examples.

The goal

You want to test the systems with **test cases** — short audio clips where you know the right answer. Then for each test case, you check whether the system identifies it correctly.

A test case = (short audio clip, known correct answer). In the literature, this is called a **query**.

Making ONE query, step by step

Pick a Saraga track at random. Say it's *Track 47: Sanjay Subrahmanyan performing Mohanam, 30 minutes*

long.

Pick a random offset within that track. Say 8 minutes in.

Cut a **10-second clip** starting at 8:00.

```
Track 47 (full 30-minute recording):  
[  
    ↑ 8:00 mark  
  
    == 10 sec clip ==  
]  
  
This 10-second clip is the query.  
The truth = (Track 47, 8:00).
```

That's **one query**: one 10-second clip + the answer (which track + which offset).

Doing this 1,632 times

You repeat this process **1,632 times** with different random tracks and offsets, producing 1,632 distinct 10-second audio clips. Each has its own ground truth.

The 1,632 are split into:

- **1,000 "main" queries** — totally random tracks and offsets (500 from Hindustani, 500 from Carnatic)
- **632 "ablation" queries** — offsets chosen to land on specific section types (alaap / composed / tani) so you can test per-section performance

Why two splits? The main set tests **average performance**; the ablation set lets you ask **per-section questions** ("does the model fail more on alaap than on composed?").

So far you have 1,632 audio files, all 10 seconds long.

The × 4 part: same query at four different lengths

Now the clever part. Instead of testing only 10-second clips, you want to test at multiple lengths to see how the systems handle short clips.

For each 10-second query, you make **shorter versions by trimming**:

```
Original 10-sec query:    == 10 sec ==  
  
Make a 5-sec version:    === 5 sec ===    (same start, just shorter)  
  
Make a 3-sec version:    == 3 sec ==  
  
Make a 1-sec version:    == 1 sec ==
```

All four versions start at the same point in the source track. They're just truncated differently.

The truth stays the same for all four versions (still Track 47, 8:00 — the start hasn't changed).

So **one query produces four audio files** (1 sec, 3 sec, 5 sec, 10 sec).

What's a "cell"?

A **cell** = one (query, length) combination. Each cell is one test case the system has to handle.

- Cell 1: query_001 at 1 sec
- Cell 2: query_001 at 3 sec
- Cell 3: query_001 at 5 sec
- Cell 4: query_001 at 10 sec
- Cell 5: query_002 at 1 sec
- Cell 6: query_002 at 3 sec
- ...

For each cell, the system returns its top guess; you check if it matches the truth.

The multiplication

1,632 unique queries × 4 length variants per query = 6,528 total cells.

That's the number of individual test cases each system has to handle.

For each system you run (Olaf, Dejavu, Panako, NAFP, recipe v3, NMFP), you get 6,528 right-or-wrong outcomes. Six systems × 6,528 cells = 39,168 individual matching operations total in your benchmark.

Why the headline tables only show 8 numbers per system

In the headline tables, you don't report each cell individually. You **group** cells by (split, length):

```
All cells from main split + 1-sec length      = 1,000 cells → one HR@1 number
All cells from main split + 3-sec length      = 1,000 cells → one HR@1 number
All cells from main split + 5-sec length      = 1,000 cells → one HR@1 number
All cells from main split + 10-sec length     = 1,000 cells → one HR@1 number
All cells from ablation split + 1-sec length  = 632 cells → one HR@1 number
All cells from ablation split + 3-sec length  = 632 cells → one HR@1 number
All cells from ablation split + 5-sec length  = 632 cells → one HR@1 number
All cells from ablation split + 10-sec length = 632 cells → one HR@1 number
```

That gives you **8 cell-groups** per system — the 8 numbers in the headline table.

So when the doc says "8 evaluation cells," it's the 8 groups (the headline numbers). When it says "6,528 evaluation cells," it's the individual test cases.

Same word, two slightly different meanings depending on context. The total count of 6,528 is what determines the *power* of your statistical tests; the 8 grouped numbers are what you report.

The full picture in one diagram

```
357 Saraga tracks (your music library)
|
| pick random tracks + offsets,
| cut 10-second clips, record ground truth
▼
1,632 unique 10-second queries
```

```

    | (= 1000 main + 632 ablation)
    |
    | chop each into 4 lengths (1s, 3s, 5s, 10s)
    ▼
6,528 individual test cases ("cells")
    |
    | for each, run the system, check if top-1 = truth
    | then group by (split, length)
    ▼
8 grouped HR@1 numbers per system
(this is what shows up in headline tables)

```

Concrete example with three queries

To make it stick:

query_id	source	offset	truth
h_t0034_q1	Hindustani Track 34	123.4 sec	t0034
c_t0089_q2	Carnatic Track 89	45.2 sec	t0089
h_t0034_q3	Hindustani Track 34 (again)	567.8 sec	t0034

For each of these 3 queries, you have 4 audio files:

- h_t0034_q1_1s.wav, h_t0034_q1_3s.wav, h_t0034_q1_5s.wav, h_t0034_q1_10s.wav
- c_t0089_q2_1s.wav, c_t0089_q2_3s.wav, c_t0089_q2_5s.wav, c_t0089_q2_10s.wav
- h_t0034_q3_1s.wav, h_t0034_q3_3s.wav, h_t0034_q3_5s.wav, h_t0034_q3_10s.wav

So these 3 queries produce $3 \times 4 = 12$ cells. Multiply by 1,632 queries → **6,528 cells**.

Reproducibility built in

Every random choice in the query generation has a seed:

```

# configs/test_queries_v2.yaml
seed_main: 20260420
seed_ablation: 20260421
n_queries_main_per_subcorpus: 500
sample_rate: 16000
mono: true
bit_depth: 16

```

Anyone with this config + the Saraga audio can regenerate the **exact same 1,632 queries**, byte-for-byte. Re-running your manifest script tomorrow produces identical files. That's reproducibility.

Questions they'll ask

Q: Why a 10-second clip first, then truncate? Why not cut 1-second clips directly? Two reasons: (1) Comparability — the 1, 3, 5, 10 second versions are all from the same starting point, so they're directly comparable across lengths. (2) NAFP-paper convention — the original paper does it

this way, keeping you apples-to-apples for comparison.

Q: Why is the 1-second version the *first* second of the 10-second clip, not a random second within it? Convention from the NAFP paper (the "leading-N" approach). It's a known limitation of your benchmark — documented in the README. Ideally future work re-cuts the 1-sec variant at a random offset within the 10-sec clip.

Q: Why 500-500 split between Hindustani and Carnatic? Balanced sampling. With 108 Hindustani and 249 Carnatic refs, fully-random sampling would give ~70% Carnatic queries. Forcing 500-500 makes sure both traditions are equally represented.

Q: 1,632 sounds arbitrary — why not 1,500 or 2,000? 1,000 main + 632 ablation. The 632 came naturally: it's the count of valid (track, section) combinations in Saraga where you can fit a 10-second clip into a single section. The 1,000 is a round number for the main set.

Part 11 — How you score the systems

Each system gives you predictions; you turn them into numbers. Here's how.

What the system returns

For each query, the system returns a **ranked list of candidates**, like:

```
Query: hindustani_t0034_q1_1s.wav (truth = t0034)
```

```
System returns:
```

```
1. t0034 (score 0.97) ← correct!
2. t0088 (score 0.61)
3. t0021 (score 0.58)
4. t0177 (score 0.52)
5. t0034 (score 0.50)
...
```

You compare the top match against the truth. If they match, the system got it right. If not, it got it wrong.

HR@1 — the main metric

The most important number, the one in your headline tables.

HR@1 = (number of queries where the system's #1 guess was correct) / (total queries)

Examples:

- 995 correct out of 1000 → HR@1 = 0.995 = 99.5%
- 745 correct out of 1000 → HR@1 = 0.745 = 74.5%

This is the practical metric. In an interactive app like Shazam, only the top result matters. The user doesn't scroll a list.

Other metrics you compute (but don't headline)

HR@5 = was the truth in the top 5? Looser than HR@1. Useful for systems that display a list to the user.

MRR (Mean Reciprocal Rank) captures "even when wrong, how close was the truth?" If truth was rank 2, contribute 1/2. If rank 5, contribute 1/5. Average over all queries. Captures graceful failure.

top1_near is HR@1 with an extra requirement: the predicted offset has to match the truth offset within 0.5 seconds. For sequence-level systems (NAFP, NMFP), this distinguishes "right song wrong place" from "right song right place."

For your headline, HR@1 is the number that matters.

Wilson 95% confidence interval

Single point estimates are misleading. If you measure HR@1 = 0.995 on 1000 queries, the true HR@1 could be anywhere from ~0.989 to ~0.998. The **Wilson confidence interval** gives that range.

A 95% CI of [0.989, 0.998] means: with 95% confidence, the true HR@1 is between 0.989 and 0.998. Two systems whose CIs overlap can't be confidently ranked from just point estimates — you need a paired statistical test (see Part 14's mention of McNemar).

Every scores.json file in your results has CIs attached:

```
{
  "hr@1": 0.995,
  "hr@1_ci_low": 0.989,
  "hr@1_ci_high": 0.998
}
```

The "is the improvement real?" question

Two systems on the same queries, one slightly better. How do you know it's a real improvement and not luck?

You use a **paired statistical test** (McNemar's test, see the deep guide for math). It asks: "given both systems saw the same queries, is the gap between them too big to be coincidence?"

For your recipe v3 on main_1s: pooled p-value = 3.18×10^{-6} , meaning roughly 1 chance in 300,000 that the improvement is luck. **Statistically, the improvement is real.**

The Bonferroni correction (also in the deep guide) makes the bar stricter because you tested 8 cells. Your p-value passes even the stricter bar by a factor of ~1900.

For now just remember: the improvement is statistically validated, not eyeballed.

Questions they'll ask

Q: Why HR@1 and not "accuracy"? Same idea, different name. HR@1 is the standard name in the retrieval literature; "accuracy" is the term in classification. The convention is HR@1 for this task.

Q: Why not precision and recall? Precision/recall are for open-world retrieval where some queries shouldn't be matched. In your closed-world benchmark (truth always in library), every query SHOULD be matched, so the simpler HR@1 is appropriate. In the open-world demo, you do report TPR/FPR (Part 19).

Q: How do you know the eval code doesn't have a bug? Three layers of defense: (1) the code is public on GitHub, anyone can audit; (2) the result parquets are public on HF, anyone can re-aggregate; (3) cross-seed consistency — 3 independent training runs all give similar numbers (± 2 misses), which would be unlikely if there was a code bug.

Part 12 — Seeds and why you used three

I noticed this confused you. Let me teach it properly.

What a seed is

A **seed** is a number that you give to a random number generator. Same seed → same sequence of "random" numbers, every time.

```
seed = 42 → random numbers: 0.37, 0.91, 0.08, 0.55, ...
seed = 42 → random numbers: 0.37, 0.91, 0.08, 0.55, ... (identical!)
seed = 137 → random numbers: 0.62, 0.14, 0.79, 0.23, ... (different)
```

Computers can't make truly random numbers. They use math to fake it. The seed is the starting input.

Why is this useful? Because it makes your work reproducible. If you train your model with seed 42 today, and someone else trains your model with seed 42 next year, they get the *exact same trained model*. They can verify your numbers.

Where seeds plug into training

At the very start of any training script, three lines set the seed:

```
SEED = 42

import random
random.seed(SEED)           # Python's random module
import numpy as np
np.random.seed(SEED)        # NumPy's random
import tensorflow as tf
tf.random.set_seed(SEED)     # TensorFlow's random
```

From this point on, *every random thing in the code* is determined by the seed.

What becomes random because of the seed

Many things during training need random numbers. The seed locks all of them down:

1. Initial model weights. When you create a fresh NAFB model, every layer's filter weights start as random small numbers. With seed 42, those 19 million starting values are one specific set. With seed 137, they're a different set.

2. Batch order. Each training epoch shuffles the training data. The seed locks the shuffle order. Same seed

→ same epoch-1 order, same epoch-2 order, etc.

3. Which audio segments are picked. For each track, the sampler picks a random 1-second offset to use as that batch's anchor. The seed determines those offsets.

4. Augmentation choices. For each positive sample, the augmenter picks:

- Which noise clip to mix in
- What SNR to mix at
- Which impulse response to use
- Where to put the SpecAugment cutout

All seeded.

5. (Hypothetical) dropout patterns. If the model had dropout layers (NAFP doesn't, but many models do), the random neuron silencing would also be seeded.

Net result: train with seed 42 today vs seed 42 next month, you get **byte-identical** trained model weights. Reproducibility solved.

Why you used three seeds (not one)

Because one seed could be lucky.

Imagine seed 42 happens to land in a *really good* spot — the random initial weights happen to be perfect for Indian classical music, just by chance. If you only trained once with seed 42 and reported great results, you couldn't tell whether your recipe is genuinely better or whether seed 42 was just lucky.

So you trained recipe v3 **three separate times** with three different seeds. Each was an independent training run from scratch:

- **Seed 42** → trained model #1 → 0.993 HR@1 on main_1s
- **Seed 137** → trained model #2 → 0.997 HR@1
- **Seed 2026** → trained model #3 → 0.995 HR@1

All three beat the baseline (0.983). That's not luck — that's the recipe genuinely working.

The mean across seeds: **0.995**. Standard deviation: ~2 misses. Stable.

Why those specific numbers (42, 137, 2026)

You picked the seeds **before training** and committed them to git in the protocol. Three numbers, each with a small story:

- **42** — the universal random-seed cliché (Hitchhiker's Guide to the Galaxy reference; almost every ML codebase uses 42 somewhere)
- **137** — a prime number (a habit in ML/physics — primes "feel more random")
- **2026** — the current year

The specific values are irrelevant. What matters is that they were committed *before training*, so no one can accuse you of trying 50 seeds and reporting the lucky ones.

Seeds for OTHER things in your project

Quick aside — "seed" appears in multiple places in your project, doing different jobs:

Seed value	Where used	What it locks down
42, 137, 2026	Recipe v3 training	Model weights, sampling, augmentation
20260420	Main query manifest	Which 1000 tracks/offsets get sampled
20260421	Ablation query manifest	Which 632 section-aligned offsets get sampled
20260515	Threshold calibration	Which 200 FMA clips become OOL probes

These are different "random generators" doing different jobs. They don't interfere.

Questions they'll ask

Q: Why three seeds, not five or ten? Three is the minimum for a "pooled" statistical test. Five would give tighter confidence intervals. Ten would be overkill for B.Tech-II. Three was time-budget-limited (each training run is 27 min + 2 hours evaluation; three seeds = a full day of compute).

Q: If you picked seeds in advance, how do I know you didn't try other seeds and just pick the 3 best? Because the seeds are in PROTOCOL.md, committed to git BEFORE training started. The commit timestamp is in the git history. Anyone can `git log` and verify.

Q: What if seeds 42, 137, 2026 all happen to be lucky by coincidence? Possible but extremely unlikely. The pooled McNemar p-value of 3.18×10^{-6} accounts for this — it's saying "across 3000 paired observations, the improvement signal is overwhelmingly strong." If all 3 seeds were just lucky, you'd need to draw 3 lucky seeds in a row, which is at the 1-in-millions level.

Part 13 — Kaggle baseline training

The starting point: training your baseline NAFP model from scratch on Kaggle.

Why Kaggle?

Kaggle gives free GPU access (Tesla T4, 15 GB VRAM) with a 30-hour weekly quota and 12-hour per-kernel limit. This makes it the natural choice for academic/student work — no cost, sufficient compute.

The alternative was Colab Free, but Colab's GPU access is less consistent (sometimes T4, sometimes nothing, hard to predict).

Training data: FMA-medium 10k

You trained on **FMA-medium 10k_icassp** — 10,000 Western tracks at 30 seconds each. This is the same training subset the original NAFP paper used.

Why Western music when you're testing on Indian classical?

1. **Convention.** The NAFP paper uses FMA-medium. Using the same data gives a clean baseline comparison.
2. **Public availability.** FMA-medium is freely available under Creative Commons. No license issues.

3. **Cross-domain transfer.** Contrastive learning's representations are genre-agnostic. The model learns "what makes audio identifiable" in general — this transfers to Indian classical without retraining.

4. **Indian classical training data is scarce.** Saraga at 357 tracks is too small for training; it's enough for testing.

The cross-domain transfer is actually a finding of your project: a model trained only on Western music works well on Indian classical because the features it learns are universal.

The training config

NAFP defaults, with one accommodation for the 12-hour Kaggle limit:

Hyperparameter	Value
Sample rate	8 kHz
Batch size	120
N anchors per batch	60
Loss	NT-Xent ($\tau = 0.05$)
Optimizer	Adam, LR 1e-4 with cosine decay
Sampler	seg_mode = all (NAFP default)
F_MIN (mel low cutoff)	300 Hz (NAFP default)
Max epochs	10 (instead of paper's 100)
Embedding size	128
Training duration	~45 minutes on T4

The big difference from the paper: 10 epochs instead of 100. Why?

- 100 epochs at the paper's batch size would take ~7.5 hours on T4 — too close to the 12-hour kernel limit for safety
- NT-Xent loss converges fast on FMA-medium; by epoch 10 you've captured ~80% of the gain
- 10 epochs makes the baseline reproducible by anyone with a free Kaggle account in under an hour

The trade-off is documented: your baseline is "NAFP architecture at 10 epochs on FMA-medium," not "the NAFP-paper published checkpoint." This is honest framing.

The gotcha you had to handle: Kaggle phone verification

Kaggle requires **phone-verified accounts** to get full GPU access. Unverified accounts get *silently downgraded* to CPU without any error message.

Your account (`aboutpritam`) is unverified. So your training jobs would say "running on GPU" but actually be on CPU — which would take **75 hours instead of 45 minutes**.

You added a fail-fast check at the start of training. It does a 2048×2048 matrix multiplication and times it:

```
import time
import tensorflow as tf
with tf.device("/GPU:0"):
    a = tf.random.normal([2048, 2048])
    t0 = time.time()
    for _ in range(10):
        b = tf.matmul(a, a)
        _ = b.numpy()
    elapsed = time.time() - t0
    assert elapsed < 0.5, f"GPU too slow ({elapsed}s) - likely on CPU. Abort."
```

Healthy T4: < 30 ms. CPU: > 500 ms. The assertion fires before any compute is wasted.

This kind of fail-fast check is now a permanent memory note for any future training scripts. Saved hours.

What the baseline produced

After 45 minutes of training, you got **ckpt-10** — the 10th-epoch checkpoint. About 80 MB on disk. You downloaded it to your Mac and ran the evaluation:

Cell	HR@1
main_1s	0.983 (17 miss / 1000) ← the cell you'll focus on
main_3s	0.998 (2 miss)
main_5s	0.999 (1 miss)
main_10s	1.000 (0 miss)
ablation_1s	0.979 (13 miss / 632)
ablation_3s	1.000
ablation_5s	1.000
ablation_10s	1.000

Two cells with room to improve: main_1s and ablation_1s. Both at 1-second clips. The other 6 cells are at the ceiling.

Those 17 misses on main_1s become the focus of your recipe v3 improvement.

Questions they'll ask

Q: Why not train longer, get a stronger baseline? Time budget. The baseline is a *reference point*. If you over-tune the baseline, the comparison with recipe v3 gets muddier (is the gain from recipe or just from training longer?). 10 epochs is a clean reference.

Q: Is your baseline weaker than the NAFP paper's published checkpoint? Probably slightly. Your baseline trained 10 epochs vs the paper's 100. The paper might score marginally higher on this benchmark. But: you compare *your* baseline to *your* recipe v3 (apples to apples, same training compute), so the comparison is fair.

Q: What does "checkpoint" mean? A snapshot of the model's weights at a specific training step.

ckpt-10 is the model after 10 training epochs. To use the model later, you load these weights into a fresh NAFP architecture. Like saving a game state.

Part 14 — Recipe v3: your improvement

The headline contribution. Let me walk through what you did and why each piece mattered.

What you changed (the recipe diff)

Five changes from the baseline, all carefully chosen:

Knob	Baseline	Recipe v3	Source of idea
F_MIN (mel filterbank low cutoff)	300 Hz	160 Hz	NMFP fix #5
Sampler (per-batch segment selection)	seg_mode=all	random_oneshot	NMFP fix #2
Batch size	120	320	Scaled up for more NT-Xent negatives
N anchors per batch	60	160	Scaled with batch size
Max epochs	10	30	More epochs to let the recipe compound

Everything else (loss function, optimizer, training data, architecture) stayed the same on purpose — to isolate the effect of these specific changes.

Why each change, in plain English

F_MIN: 300 → 160 Hz

The NAFP baseline's mel filterbank ignores everything below 300 Hz. Anything quieter or lower in frequency than 300 Hz is invisible to the model.

But Indian classical music has critical content below 300 Hz:

Source	Frequency range
Tanpura drone (the continuous low hum)	100-200 Hz
Male vocal fundamentals (bass-baritone)	80-200 Hz
Female vocal fundamentals (alto)	175-300 Hz
Tabla bass (lower drum)	100-200 Hz
Mridangam bass	100-200 Hz

By dropping F_MIN to 160 Hz, the model now sees this entire low-frequency band. More frequency information → more discriminative features → better at telling recordings apart.

Concretely: a baseline model might miss subtle differences in how two singers' tonic frequencies sound; the recipe v3 model catches them.

Sampler: `seg_mode=all` → `random_oneshot`

I covered this in Part 6 but it's worth re-stating because it's one of the two key fixes.

The default `seg_mode=all` sampler can put two segments from the **same track** into one training batch. NT-Xent treats them as negatives — pushes them apart in fingerprint space. But they're from the same recording! They should be similar. The loss is sending the wrong signal.

The fix `random_oneshot` ensures at most ONE segment per track per batch. Reshuffles each epoch. No false negatives.

In Indian classical music, this matters extra. Same-track segments are particularly similar (same voice, same room, similar musical material). The false-negative penalty hurts more here than on FMA's diverse pop. Fixing it has outsized impact.

Batch size: 120 → 320

NT-Xent benefits from more **negative examples per anchor**. With batch 120 (60 anchors + 60 positives), each anchor sees 119 negatives. With batch 320 (160 anchors + 160 positives), each anchor sees 319 negatives. Roughly 3× more "what's NOT this anchor" information per training step → sharper discrimination.

NMFP used batch 1536 (~13× more than baseline). You used 320 — the max your Colab L4 GPU can hold in 24 GB VRAM. Constrained but still much better than 120.

Max epochs: 10 → 30

Three changes (sampler, `F_MIN`, batch size) need time to compound. At 10 epochs, the sampler hasn't reshuffled enough times, the low-frequency info hasn't been fully integrated. 30 epochs gives convergence headroom.

The trade-off is acknowledged in your protocol: "improvement is recipe + epoch budget, not recipe alone." You're not claiming "this is just the recipe."

Why three seeds (42, 137, 2026)

Already covered in Part 12. To recap: you trained recipe v3 three independent times with three different random seeds. All three beat baseline, with mean HR@1 = 0.995 and std ~2 misses. The improvement is consistent, not seed-lucky.

Where you trained: Colab L4

Why Colab L4 instead of Kaggle T4?

- Colab L4 (24 GB VRAM) is more powerful than Kaggle T4 (15 GB)
- L4 fits batch size 320 in memory; T4 maxes out around 200
- L4 trains ~25% faster per step than T4
- Colab GPU access is per-session, no weekly quota

Wall time per seed: 27 minutes. Three seeds: 80 minutes total.

Public training notebook: https://colab.research.google.com/drive/1bS3q8vOiyIqW2IyICls_i-cGFKC8VZc6.

Open it in Colab, switch to an L4 runtime, and "Run all" produces the same `ckpt-30` (per seed) that's deployed in the demo.

The pre-registration discipline

The critical methodological step that makes your result defensible.

Before training started, you wrote `data/results/nafp/recipe_v3_30ep/PROTOCOL.md` and committed it to git. The protocol looked:

- The exact hyperparameters (`F_MIN=160`, `sampler=random_oneshot`, `BSZ=320`, `NA=160`, `MAX_EPOCH=30`)
- The three seeds (42, 137, 2026)
- The primary endpoint (HR@1 on main_1s, pooled McNemar test, Bonferroni-corrected)
- The falsification rule (if any cell regresses by > 1 miss on average, the recipe fails)

This is committed in git, so the commit timestamp predates the training. A reviewer can `git log -- PROTOCOL.md` and verify the protocol was written *before* the results.

This means you cannot be accused of cherry-picking, p-hacking, or moving goalposts. The prediction was made; the data was collected; the test was applied as specified. This is the gold standard for empirical research.

Questions they'll ask

Q: Why these 5 changes and not others? The first two (`F_MIN`, `sampler`) are NMFP-paper-validated fixes that don't require code changes — just config flips. The batch size and epoch changes scale the training to use more compute, which amplifies the recipe's effect. Together they're the most "bang per line of code" you could get.

Q: How do you know the improvement is from the recipe and not just from training longer? Honestly, you can't fully separate them without more ablation runs. Your README is explicit: "improvement is recipe + epoch budget, not recipe alone." Future work would run recipe v3 at 10 epochs vs baseline at 30 epochs to disentangle the contributions.

Q: What's a "pre-registered" protocol? A document committed to public storage (git, OSF, etc.) BEFORE the experiment runs, specifying what you'll test and how. Prevents after-the-fact rationalization ("oh I meant to test that all along!"). Standard practice in medicine and increasingly in ML.

Part 15 — The headline result

What did all this work produce? Let me put the numbers up front and then explain how to interpret them.

The big table

System	1 sec HR@1	3 sec	5 sec	10 sec
--------	------------	-------	-------	--------

Olaf	49.2%	95.5%	99.3%	99.8%
Dejavu	74.5%	96.9%	99.4%	100%
Panako	0.0%	0.0%	92.2%	99.7%
NAFP baseline	98.3%	99.8%	99.9%	100%
Recipe v3 (your work)	99.5%	100%	100%	100%
NMFP ceiling	100%	100%	100%	100%

What this table tells you

Read it left to right and you see the **length effect**: every system gets better as the query length grows.

Read it top to bottom and you see the **system effect**: at every length, neural systems are at least as good as classical systems.

The interesting story is in the top-left quadrant — short clips with classical systems. That's where the systems differ most.

The error-rate reduction view

Percentages can be misleading. 98.3% → 99.5% sounds small. But what does it mean in mistakes?

Cell	Baseline misses	Recipe v3 misses	Reduction
main_1s (1000 queries)	17	5 (3-seed mean)	−70.6%
ablation_1s (632 queries)	13	5.7	−56.2%
main_3s (1000 queries)	2	0	−100%
main_5s (1000 queries)	1	0	−100%
Other 4 cells	0 each	0 each	—
All 8 cells combined	33 / 6528	10.7 / 6528	−67.6%

About **two-thirds fewer mistakes** across the entire benchmark.

Why this matters in practice

For a hypothetical Indian-classical Shazam app:

- With the baseline model, 17 in 1000 users get a wrong answer to a 1-second query
- With recipe v3, only 5 in 1000 get a wrong answer

If the app handles 1 million queries per day:

- Baseline: 17,000 wrong answers per day
- Recipe v3: 5,000 wrong answers per day
- Difference: 12,000 users per day no longer mis-served

In ML research, "1.2% improvement" sounds boring. In practice, it's "12,000 users per million per day not getting a wrong answer."

Comparison to NMFP

NMFP scores 100% on every cell. Recipe v3 is at 99.5% on the hardest cell.

Gap to NMFP: 0.5% HR@1, or 5 misses per 1000 on the hardest cell.

Compute to get there:

- NMFP: 100 epochs at batch 1536 (~13 hours equivalent on L4)
- Recipe v3: 30 epochs at batch 320 × 3 seeds (1.3 hours total)

Recipe v3 reaches ~95% of NMFP's improvement at ~10% of their compute.

The framing matters: **you don't beat NMFP**. You demonstrate that 2 of their 5 fixes capture most of the practical gain at a fraction of the cost.

Is the improvement statistically real?

This is where the statistics come in. Three independent training runs (seeds 42, 137, 2026) all beat baseline. Pooled across them on main_1s, the McNemar p-value is 3.18×10^{-6} — about 1 chance in 300,000 that this is luck.

Even after the Bonferroni correction for testing 8 cells, the result clears the strict bar by ~1900×.

Both main_1s and ablation_1s — the two hardest cells — are Bonferroni-significant. The other 6 cells are at the ceiling (no room to improve, no statistical signal).

In plain English: the improvement is statistically real, not lucky.

Questions they'll ask

Q: 99.5% vs 98.3% — is that really a big deal? In mistakes-per-1000 terms: 17 → 5. A 70% reduction in errors. For any product handling millions of queries, that's a huge real-world improvement.

Q: Why didn't recipe v3 beat NMFP? Because NMFP applied all 5 fixes and trained ~10× longer at ~4× larger batch. You applied 2 of their cheapest fixes at much less compute. You closed most of the gap, not all of it.

Q: Could you reach 100% with more work? Probably, with the remaining 3 NMFP fixes (#1, #3, #4) plus Triplet loss. Those are coded as future work. The 2 persistent misses on main_1s are particularly stubborn — same-artist same-section confusions on very short clips — and might need fundamental architectural changes.

Part 16 — Why baseline made mistakes (the failure-mode story)

This part is one of the most interesting findings of your project. You didn't just measure how many mistakes baseline made — you investigated *what kind* of mistakes they were.

Going through the 17 baseline misses

After running baseline on main_1s, you had a list of 17 queries it got wrong. For each one, you looked at:

- What was the truth (correct ref)
- What did baseline predict
- What's the relationship between them

The pattern that emerged:

Type of mistake	Count
Same-artist different-track	14 (82%)
Cross-corpus (Hindustani query → Carnatic prediction or vice versa)	1
Twin pair (different recording of same composition)	1
Other	1

82% of mistakes are the same kind: the model picked a different recording, but it's by the same artist as the correct answer.

Why same-artist confusion happens

The model isn't randomly wrong — it's wrong in a *systematic* way. Let me explain why.

At 1-second clips, melody hasn't fully unfolded yet. There's not enough musical content to identify the raaga or composition uniquely. So the model falls back on **timbre** — the "color" of the voice or instrument.

But timbre is very **artist-consistent**. Sanjay Subrahmanyam's voice sounds like Sanjay Subrahmanyam whether he's singing Mohanam or Bhairavi. The vocal qualities — vibrato pattern, resonance, attack character — are stable across his recordings.

So at 1 second the model essentially says: "this sounds like Sanjay Subrahmanyam singing." It's right about the artist. But it's wrong about *which specific Sanjay Subrahmanyam recording* this clip is from.

It's getting the raaga right (often) but the recording identity wrong.

Why this is a "Indian classical specific" problem

In Western pop, an artist usually puts out a few albums with relatively distinct songs. Two recordings by the same artist sound noticeably different (different songs, different production styles).

In Indian classical, an artist might have dozens of recorded concerts performing many overlapping raagas. The "same artist performing the same raaga in two different concerts" case is much more common than in pop. Same voice, same room (often the same recital hall), similar musical material → very similar audio → confusion.

This is why your failure-mode finding is genre-specific. The same model on Western pop wouldn't exhibit this pattern as strongly.

How recipe v3 helps

Recipe v3 recovers **15 of the 17 baseline misses across all 3 seeds**. The mechanism:

1. **random_oneshot sampler** stops the model from being forced to push same-track segments apart during training. This frees up "headroom" in the embedding space to encode within-artist variation — i.e., the differences between Sanjay's Mohanam vs Sanjay's Bhairavi.
2. **F_MIN = 160 Hz** adds more low-frequency information to each fingerprint. The tanpura drone and vocal fundamentals vary between recordings even when the artist is the same (different concerts have different ambient setups). The model can use this additional information for discrimination.

Together: recipe v3 has more capacity to encode within-artist differences, and more information to draw on.

What recipe v3 still misses

Two queries persist as misses across all 3 seeds. Looking at them:

Both are same-artist, same-section confusions on very short Carnatic composed sections. The hardest possible case. Probably need:

- Longer queries (recipe v3 already hits 100% at 3 seconds — these particular queries only fail at 1 sec)
- NMFP fix #4 (1-second acoustic history) which gives the model more context
- Genuinely better training data for Indian classical

Cross-seed consistency

Just to drive home that the improvement is real:

Seed	main_1s misses
42	7 (HR@1 = 99.3%)
137	3 (HR@1 = 99.7%)
2026	5 (HR@1 = 99.5%)
Mean	5 (HR@1 = 99.5%)
Standard deviation	~2

Three seeds, three different miss counts, but all far below baseline's 17. Stable, real improvement.

Questions they'll ask

Q: How did you identify "same-artist confusions"? Cross-referenced the predicted track ID with the truth track ID. Looked up both in Saraga metadata. If the `artist` field matched, it's a same-artist confusion. Documented in `docs/POST_MORTEM_RECIPE_V3.md`.

Q: Same-artist confusion sounds intuitive. Why is it a finding? Because no one had quantified it before for Indian classical AFP. The finding tells future researchers: "if you want to improve here, focus on within-artist discrimination, not on raaga modeling." That's actionable guidance.

Q: Doesn't that mean your model still has problems? Yes — 2 persistent misses remain. The improvement is partial, just statistically significant. Honest about it.

Part 17 — Publishing on Hugging Face

Why and how you released everything for public consumption.

The Hugging Face dataset

You published your benchmark as a public Hugging Face dataset at:

[Tachyeon/audio-fingerprint-indian-bench \(v0.6\)](#)

About 530 MB total across 1,797 files.

What's on it

```
data/
├─ queries/                ← 1000 main queries × 4 lengths = 4000 WAV files
├─ queries_ablation/      ← 632 ablation queries × 4 lengths = 2528 WAV files
├─ refs.parquet           ← 357-row metadata for Saraga references
├─ results/               ← Every system's predictions
│   ├─ olaf/, dejavu/, panako/
│   ├─ nafp/saraga_only_*  ← baseline results
│   ├─ nafp/nmfp_eval/    ← NMFP ceiling results
│   ├─ nafp/recipe_v3_30ep/ ← your improvement
│   │   ├─ PROTOCOL.md    ← pre-registration
│   │   ├─ RESULTS.md     ← final report
│   │   └─ pooled_mcnemar.csv ← primary endpoint
│   └─ seed{42,137,2026}_eval/
└─ threshold_calibration/
├─ inspection/            ← library audits (e.g., twin detection)
└─ configs/               ← reproducibility configs
```

Every result parquet has per-query data: query_id, predicted rank, predicted ref_id, predicted offset, score, is_correct. Anyone can re-aggregate to verify your numbers.

What's NOT on it

- Source Saraga MP3s (CC-BY-NC-SA restricts redistribution; users must get from Zenodo)
- Source FMA audio (10 GB, not your data to redistribute)
- Model checkpoint binaries (those live on the demo Space instead)

The README explains exactly what to download from where.

Why publish at all?

Four reasons:

1. **Reproducibility** — anyone can inspect every number you claim
2. **Replication** — another researcher can run their own AFP system on YOUR exact queries
3. **Citability** — gives a stable identifier for your work
4. **Transparency** — pre-registration protocols are one click away

The dataset card (the README on the HF dataset page) walks users through what they're seeing, how to load it, and how to verify your numbers.

How users actually use it

```
# Load every result parquet
from datasets import load_dataset
ds = load_dataset("Tachyeon/audio-fingerprint-indian-bench", "results")

# Or download a specific file directly
from huggingface_hub import hf_hub_download
p = hf_hub_download(
    "Tachyeon/audio-fingerprint-indian-bench",
    "results/recipe_v3_30ep/pooled_mcnemar.csv",
    repo_type="dataset",
)
import pandas as pd
print(pd.read_csv(p))
```

That's it. Anyone with `pip install datasets` can audit your headline numbers in 5 minutes.

The GitHub code repo

Separately, your code is on GitHub at:

github.com/ipritamdash/afp-indian-classical (private, MIT license)

About 250 files, 3 MB of code. Includes:

- Every training script (baseline, recipe v3)
- Every evaluation script (Olaf, Dejavu, Panako, NAFP, NMFP)
- Every analysis script (post-mortem, threshold calibration, McNemar pooling)
- The Gradio demo app + redeployment scripts
- All YAML configs

Anyone can clone, run the download scripts (for Saraga + FMA), and reproduce the entire pipeline end-to-end.

Questions they'll ask

Q: Why publish if no one cares about a B.Tech-II project? Because publishing makes the work *defendable*. Anyone can audit. You're not relying on people taking your word for it.

Q: What if Saraga changes their license? Then you'd need to re-evaluate redistribution rights. CC-BY-NC-SA is currently the license. Your published artifacts (queries, results) inherit it.

Q: Why MIT for code but CC-BY-NC-SA for the dataset? Code is yours (MIT lets anyone use it). The dataset inherits Saraga's license (CC-BY-NC-SA — non-commercial). These aren't the same kind of artifact and have different appropriate licenses.

Part 18 — The live demo

The most user-facing piece of the project. Anyone with a web browser can try recipe v3 on their own audio.

The URL

<https://huggingface.co/spaces/Tachyeon/afp-indian-classical-demo>

What the user sees

1. A web page with a file uploader and a microphone button
2. They drag-drop or record an audio clip (1-30 seconds, any common format)
3. After 5-7 seconds, they see:
 - A **verdict card** with one of three states:
 - ☐ ✓ **Match found (high confidence)** — green
 - ☒ **Match found** — yellow
 - ☐ ✕ **No match in library** — red
 - A **top-5 candidates table** with artist, raaga, taala, and offset for each

What happens under the hood

```
User uploads audio
    ↓
ffmpeg decodes → 8 kHz mono float audio
    ↓
Slice into 1-second windows (0.5 sec hop)
    ↓
Each window → 128-number signature via the recipe v3 model
    ↓
FAISS searches 690,414 pre-computed Saraga signatures
    ↓
Top-K candidates aggregated by track (mean similarity)
    ↓
Top-1 candidate's score compared against thresholds
    ↓
Display verdict + top-5 table
```

The model is **recipe v3 seed 42 ckpt-30** — the primary trained model.

Architecture

demo/	
├─ app.py	← Gradio UI (~280 lines)
├─ DEPLOY.md	← redeploy guide for others
├─ deploy.py	← one-command redeploy script
├─ README.md	← Space landing page
├─ requirements.txt	← pinned dependencies

— packages.txt	← apt deps (ffmpeg)
— artifacts/	← auto-downloaded on first run
— ckpt-30.{data, index}	← 194 MB recipe v3 weights
— ref_embs.mm	← 354 MB FAISS-ready embeddings
— ref_segment_lookup.parquet	← 5 MB segment-to-track lookup
— refs_hindustani.csv	← metadata
— refs_carnatic.csv	
— recipe_v3.yaml	← model config
— threshold.json	← calibrated T values
— upstream/	← patched NAFP source code

Total Space size: ~600 MB. Within Hugging Face free tier limits.

Hosting

- **Hosting service:** Hugging Face Spaces (free CPU tier)
- **Hardware:** Free CPU (no GPU needed for inference; the model is tiny)
- **Latency:** ~120 ms compute + ~5-7 sec end-to-end including audio upload
- **Cost:** \$0 (free tier)
- **Cold-start behavior:** Space sleeps after 48 hours of inactivity, wakes up automatically (~90 sec)

The one critical detail (TF_USE_LEGACY_KERAS)

In `app.py` line 19, before *any* TensorFlow import:

```
import os
os.environ["TF_USE_LEGACY_KERAS"] = "1"

import tensorflow as tf  # NOW it's safe
```

Why is this crucial? Without it, TensorFlow 2.19 silently **partial-restores** the NAFP checkpoint:

- The CNN backbone weights load correctly
- The DivEnc head weights fail to load (different naming convention in Keras 3)
- The model produces **random outputs** for every query
- **No error is raised** — the Space "works" but every answer is garbage

This took hours to discover during development. Now there's a permanent memory note flagging it.

The three build failures you solved

When deploying, the Space build failed three times before working:

Failure 1 — `tensorflow==2.19.0 not found` HF Spaces' new default Python is 3.13. TF 2.19 has no Python 3.13 wheels. **Fix:** Pin `python_version: "3.11"` in the README YAML frontmatter.

Failure 2 — **numpy version conflict** TF 2.19 requires `numpy<2.2, >=1.26`; kapre 0.3.7 PyPI rebuild requires `numpy>=2.0`. **Fix:** Intersection: `numpy>=2.0, <2.2`.

Failure 3 — **librosa version conflict** kapre 0.3.7 PyPI rebuild requires `librosa>=0.11`; pip's default resolution picked 0.10. **Fix:** Pin `librosa>=0.11, <1.0`.

The final pinned `requirements.txt` is non-negotiable. Document this so future-you doesn't accidentally relax the pins:

```
tensorflow==2.19.0
tf-keras==2.19.0
kapre==0.3.7
numpy>=2.0,<2.2
librosa>=0.11,<1.0
gradio==6.14.0
soundfile
faiss-cpu
pandas
pyarrow
huggingface_hub
```

Stress tests

You ran 13 manual tests against the deployed Space:

Category	n	Result
In-library Saraga clips (varied lengths + corpus)	5	5/5 correctly matched
Out-of-library FMA clips	3	3/3 correctly rejected
Known recipe-v3 miss queries	3	3/3 wrongly matched (expected, consistent with benchmark)
Silent audio	1	Rejected ("no match")
Pure white noise	1	Rejected
Too-short (0.5 sec)	1	Rejected with error message

The demo's behavior matches the benchmark numbers. Real-world deployment is consistent with reported HR@1.

Questions they'll ask

Q: Why bother with a live demo? Because numbers in a table are abstract. A working demo makes the system tangible. Anyone — your professor, your interviewer, your future self — can try it without reading code or papers.

Q: Why CPU and not GPU on Spaces? Inference takes ~120 ms on CPU. GPU is overkill and costs money. Free CPU is enough for a research demo.

Q: What audio formats does the demo accept? Anything ffmpeg can decode: WAV, MP3, M4A, OGG, WebM, FLAC, etc. Plus mic recording on most browsers.

Part 19 — Threshold calibration (open-world)

The piece that lets the demo say "I don't know" when appropriate.

The problem

Your benchmark always has the truth in the library — every query is from a Saraga track. But in the wild (the demo), users upload anything: Bollywood, voice memos, white noise, random YouTube clips. **Most of those have no match in your 357-track library.**

Without handling this, the demo would always return its best guess, even when the guess is meaningless. User uploads white noise → demo returns "matched Saraga Track 247!" with low confidence. User trusts it. Bad.

You need an "I don't know" capability. That requires a **confidence threshold**.

The threshold idea

Every match comes with a similarity score (between 0 and 1, basically cosine of the angle between the query fingerprint and the matched library fingerprint).

For:

- **In-library queries** (truth IS in library): scores tend to be HIGH (mean ~0.95)
- **Out-of-library queries** (truth NOT in library): scores tend to be LOW (mean ~0.62)

If you pick a threshold T :

- Scores $\geq T \rightarrow$ accept the match
- Scores $< T \rightarrow$ reject, say "no match in library"

The question: where should T be?

Pre-registered protocol

Before measuring anything, you wrote `data/results/threshold_calibration/PROTOCOL.md`. The protocol looks:

1. **In-library distribution:** the rank-1 scores from your 993 correctly-matched main_1s queries (recipe v3 seed 42)
2. **Out-of-library probe set:** 200 random clips from FMA-medium (Western music — guaranteed NOT in Saraga)
3. **Threshold-selection rule:** pick the smallest T such that false-positive rate (FPR) on OOL $\leq 5\%$
4. **Feasibility check:** if no T achieves both $FPR \leq 5\%$ AND $TPR \geq 95\%$, declare calibration infeasible

This protocol was committed to git BEFORE measuring the OOL scores. Anyone can audit.

Measuring the OOL distribution

The OOL probe script does this:

- Sample 200 FMA-medium clips with seed 20260515 (reproducible)
- For each: load 1 sec at 8 kHz mono, encode through recipe v3, FAISS-search Saraga library, record top-1 score

- 33 clips were dropped (bad MP3s, too short) → 167 valid OOL scores

The results

Distribution	n	mean	p5/p95	max
In-library correct matches	993	0.948	p5=0.840	0.9997
Out-of-library FMA	167	0.616	p95=0.766	0.892

In-library scores hover around 0.95. OOL scores hover around 0.62. They overlap a bit in [0.69, 0.89], but mostly they're well-separated.

Picking T

Sweeping T from 0 to 1:

T	FPR (OOL falsely accepted)	TPR (in-library correctly accepted)
0.5	100%	100%
0.7	32%	99.9%
0.7867	4.2%	99.1%
0.85	1.2%	93%
0.8918	0%	85.4%
0.95	0%	49%

The protocol's rule (smallest T with $FPR \leq 5\%$) gives **T_default = 0.7867**. A tighter "high confidence" rule ($FPR \leq 1\%$) gives **T_high_conf = 0.8918**.

The feasibility check passes ($TPR 99.1\% \geq 95\%$).

What the demo does with these thresholds

```
T_DEFAULT = 0.7867
T_HIGH_CONF = 0.8918

if top1_score >= T_HIGH_CONF:
    verdict = "✓ Match found (high confidence)" # green
elif top1_score >= T_DEFAULT:
    verdict = "● Match found" # yellow
else:
    verdict = "✗ No match in library" # red
```

So:

- Clean Saraga clip (score 0.96) → green
- Bollywood song (score 0.58) → red
- Degraded Saraga clip (score 0.81) → yellow

Why this is methodologically clean

The threshold isn't pulled out of thin air. It was computed by a **pre-registered protocol**. Anyone can audit `PROTOCOL.md` and verify the rule was written before the data was measured. The threshold IS what the rule says it should be — not what makes the demo look best.

If you'd picked T after looking at the data, you could be accused of cherry-picking ("oh look, $T=0.85$ makes the demo look perfect"). Pre-registration prevents this.

What the threshold does NOT cover

Honestly disclosed in the demo's About tab:

- **Indian classical music NOT in your Saraga subset** (Bollywood, CompMusic CMD/HMD): might score high due to artist-hub effects. Not tested.
- **Heavily noisy phone-mic recordings**: separate failure mode, not tested.
- **Same-artist different-work confusion**: threshold doesn't help here — the score is high but the predicted ref is wrong.

These are limitations users should know about. Better to disclose than over-promise.

Questions they'll ask

Q: Why use FMA tracks as OOL probes if the model was trained on FMA? Subtle but important. "Out-of-library" means "the truth is not in the RETRIEVAL library (Saraga)." Any FMA track has no truth in Saraga by construction. Whether the encoder saw the track during training is irrelevant — calibration is about *retrieval correctness*, not training-data novelty.

Q: What if a user uploads a Bollywood song? Will the demo correctly say "no match"? Empirically, yes — Bollywood is sonically different enough that OOL scores hover around 0.6, well below the 0.79 threshold. But not 100% guaranteed; disclosed in the About tab.

Q: Why both a default and a high-confidence threshold? User experience. The high-confidence badge gives strong matches a stronger visual indicator ("definitely the right answer"). The default threshold is the operational accept/reject bar. Two visual tiers, one underlying calibration.

Part 20 — The Counter-Questions cheat sheet

The questions a sharp viva examiner, advisor, or interviewer will most likely ask. Have a one-paragraph answer ready for each.

About the project as a whole

Q: One-sentence pitch?

"I built a 5-system audio fingerprinting benchmark on Saraga 1.5 Indian classical music, identified the same-artist confusion failure mode of NAFP, applied 2 pre-registered training-recipe fixes from NMFP, and demonstrated a statistically-significant ~68% reduction in errors with everything reproducible and live."

Q: Why this topic?

Indian classical music has acoustic characteristics (long alaaps, tanpura drones, same-artist hub effects) that differ from Western pop. Most AFP research targets Western music. No one had run all 5 systems on Saraga before. The work extends the practical reach of AFP to a new musical tradition.

Q: Most surprising finding?

The dominant failure mode isn't getting the raaga wrong — it's getting the *recording identity* wrong while keeping the artist right. 82% of NAFP-baseline's 1-second errors are same-artist confusions. Genre-specific to Indian classical.

Q: What's novel?

(1) The benchmark setup (Saraga 1.5 with main + ablation splits at 4 lengths). (2) The same-artist failure-mode analysis. (3) The pre-registered recipe-transfer evaluation. (4) The calibrated open-world demo with pre-registered threshold.

About methodology

Q: Why pre-registration?

So no one can accuse me of cherry-picking results after seeing the data. The PROTOCOL.md (hyperparameters, seeds, primary endpoint, Bonferroni correction) was committed to git before training started. Anyone can `git log` and verify.

Q: Why 3 seeds, not 5?

Time budget. 3 is the floor for pooled McNemar. 5+ would give tighter confidence intervals but each seed is ~27 min train + ~2 hours eval. Three seeds was a full day; five would be two days.

Q: Why specifically those 2 NMFP fixes?

They're the cheapest changes — F_MIN is a config flip, the sampler is one parameter. The other 3 fixes need substantial code changes in the data loader. For B.Tech-II scope, the cheap fixes with the best ratio of impact to engineering effort.

Q: Could the improvement just be from training 3× longer?

Partly. README is explicit: "improvement is recipe + epoch budget, not recipe alone." Disentangling them requires more ablation runs (future work).

Q: How do you know the eval code isn't buggy?

Three defenses: code is public on GitHub (anyone can audit), results parquets are public on HF

(anyone can re-aggregate), and cross-seed consistency (3 independent seeds give similar numbers ± 2 misses) — bugs would likely cause more variability.

About technical content

Q: How does contrastive learning work, in one sentence?

"Train the model so two augmented versions of the same audio produce similar fingerprints, and two clips from different recordings produce different fingerprints."

Q: Why CNN and not transformer?

Smaller model (19M params vs 100M+), faster training, deployable on phones, NAFP-paper validated. Transformers might do marginally better but aren't worth the extra cost for this task.

Q: What is FAISS?

A library by Facebook for fast nearest-neighbor search over vector databases. Built for embedding retrieval at scale. Your 690,414 reference vectors are searched in ~ 10 ms per query.

Q: What does the McNemar test actually do?

Compares two systems on the same questions. Counts the queries where they *disagree* (one hits, one misses). Asks "is the gap between A-helps-B-hurts and A-hurts-B-helps too big to be coincidence?"

Q: Why "Bonferroni-significant"?

Because I tested 8 evaluation cells. Without correction, the chance of one false-positive significance result is $\sim 34\%$. Bonferroni divides α by 8 $\rightarrow$ bar becomes 0.00625. My pooled p of $3.18e-06$ is $\sim 1900\times$ below that bar.

About data and corpus

Q: 357 tracks is small. Doesn't that inflate accuracy?

Yes, library size is a documented limitation. With 25,000 FMA distractors added, classical systems' accuracy would drop significantly (more hash collisions). Neural systems probably keep most of their accuracy. Adding distractors is future work.

Q: Why not train on Indian classical?

(1) NAFP paper uses FMA-medium; using the same data gives a clean baseline comparison. (2) Public Indian classical training data is too small. (3) Contrastive learning transfers across genres — features are universal.

Q: License situation for the published artifacts?

Code MIT, model weights MIT (your work, trained on Creative Commons data), reference embeddings CC-BY-NC-SA (Saraga derivative — non-commercial). NMFP weights are GPLv3 (which is why you trained your own MIT-licensed model).

About results

Q: Why is the improvement "real, not luck"?

Three independent seeds (42, 137, 2026) all beat baseline. Pooled across them, the McNemar p-value is 3.18×10^{-6} — roughly 1 in 300,000 chance of being luck. Plus the *mechanism* is interpretable: the same-artist failure mode shows the recipe is fixing a specific, named problem.

Q: Why doesn't recipe v3 beat NMFP?

NMFP applied all 5 fixes and trained 100 epochs at batch 1536 — ~10× your compute. Recipe v3 used 2 of 5 fixes at 1/10 the compute. You closed ~95% of the gap to ceiling at ~10% of the compute — not all of it.

Q: Practical takeaway?

For anyone deploying NAFP on Indian classical music: apply `F_MIN=160` and the `random_oneshot` sampler. Two small changes, no architecture modification, ~70% miss reduction at 1 second.

About the demo

Q: Why does the demo need a threshold but the benchmark doesn't?

Benchmark is closed-world (truth always in library, always return top match). Demo is open-world (users upload anything, must be able to say "no match"). Threshold separates "accept top match" from "reject as out-of-library."

Q: Why FMA probes for calibration?

FMA tracks are guaranteed-not-in-Saraga (different corpus). They're ideal "out-of-library" probes. The protocol-defined rule (pre-registered) picks `T` such that FPR on these probes is $\leq 5\%$.

Q: What if a user uploads a real Saraga clip but the demo returns "no match"?

Possible but rare (TPR at default threshold is 99.1%). About 9 in 1000 in-library clips will be falsely rejected. Documented in the About tab.

About limitations and future work

Q: Single biggest limitation?

Library size. 357 refs vs millions in production. Numbers saturate near 1.0. Adding 25k FMA distractors would test scalability — top future work item.

Q: Next semester, what would you do?

(1) Add FMA distractors. (2) Apply the remaining 3 NMFP fixes. (3) Build a phone-mic noise benchmark. (4) Run more seeds (5-7) for tighter confidence intervals.

Q: What's an important pre-registered NEGATIVE result?

Per-artist mean subtraction (Intervention 2). You hypothesized that subtracting each artist's mean embedding would reduce same-artist confusion. Pre-registered, tested, falsified — mechanism didn't work. Published as a negative result. Good science.

Q: Is this publishable?

Maybe as a benchmark paper at ISMIR or a workshop submission. The novel contributions (Saraga AFP benchmark, same-artist analysis, recipe-transfer evaluation, calibrated demo) are non-trivial. Specific venue depends on framing.

Part 21 — One-page final summary

If you forget everything else, this is what to remember.

The project, in 5 sentences

1. You built a benchmark of 5 audio fingerprinting systems on the Saraga 1.5 Indian classical music corpus (357 tracks, 6,528 evaluation cells per system).
2. You found that NAFP's baseline failure mode is same-artist confusion at 1-second clips (82% of its errors).
3. You applied 2 of NMFP's 5 published recipe fixes ($F_MIN=160$ Hz + one-anchor-per-track sampler), pre-registered the protocol, and trained 3 random seeds.
4. The improvement is ~68% fewer mistakes overall (~70% on the hardest cell), statistically real (pooled McNemar $p = 3.18 \times 10^{-6}$, Bonferroni-corrected).
5. Everything is public: code on GitHub, results on a Hugging Face dataset, live calibrated demo on Hugging Face Spaces.

Five names you must know

- **NAFP** — Neural Audio Fingerprint (Chang et al., ICASSP 2021)
- **NMFP** — Neural Music Fingerprint (Araz et al., ISMIR 2025)
- **Saraga** — your test corpus (CompMusic at UPF Barcelona)
- **FMA-medium** — your training corpus (Western Creative-Commons music)

- **Recipe v3** — your improved trained model

Five numbers you must know

- **0.983** — baseline HR@1 at main_1s (17 misses)
- **0.995** — recipe v3 mean HR@1 at main_1s (5 misses across 3 seeds)
- **3.18×10^{-6}** — pooled McNemar p-value on main_1s
- **67.6%** — error reduction across all 8 cells
- **0.7867** — calibrated out-of-library threshold

Five URLs you must know

- Demo: huggingface.co/spaces/Tachyeon/afp-indian-classical-demo
- Dataset: huggingface.co/datasets/Tachyeon/audio-fingerprint-indian-bench
- Code: github.com/ipritamdash/afp-indian-classical
- Training notebook (Colab): colab.research.google.com/drive/1bS3q8vOiylqW2lyICls_i-cGFKC8VZc6
- Saraga: zenodo.org/records/4301737
- NAFP paper: arxiv.org/abs/2010.11910

Three concepts you must explain in 30 seconds

1. **Audio fingerprinting** — turning audio into a small unique signature, then matching it against a database to identify what recording it came from.
2. **Contrastive learning** — training a model so that augmented versions of the same audio produce similar fingerprints and different recordings produce different fingerprints.
3. **Pre-registration** — committing your hypothesis and protocol to git BEFORE measuring data, so cherry-picking is impossible to do retroactively.

One sentence to close

"I improved an audio fingerprinting model for Indian classical music, proved the improvement is statistically real, and published everything for anyone to verify or extend."

End of overview doc.

For the deep math + derivations (formulas, statistical proofs), see the companion [AFP_Teaching_Guide.pdf](#). For the code, see GitHub. For the data + results, see Hugging Face.

If anything here is still confusing — point at the specific spot. We iterate until it clicks.